MedEx: Senior Frontend Engineer Take-Home Exercise

Task 3: Appointment Booking across Time Zones

Thank you for your interest in joining MedEx.

MedEx is a healthcare marketplace. Customers in Thailand, Nepal and India use it to find clinics, hospitals and labs, compare health packages, lab tests and wellness treatments, and book them online. On a single visit, one customer often deals with several providers and several currencies.

This exercise is about UI and UX when the data is messy. The data we give you has the same shape as our production API, rough edges included. How you handle those edges, and how you explain your decisions, matters much more to us than how many features you finish.

How this works

- **Time:** spend about 6–8 hours. You have 5 days from the day you receive this. If you run out of time, stop and write in your README what you would do next. That counts in your favour.
- **Stack:** React with TypeScript is required. Next.js is preferred, since it's what we use, but not required. Use any component library, styling approach or state tool you like, and tell us why you chose it.
- **Data:** the task's JSON file is in the `data/` folder next to this brief. All of it is fictional. Don't import the JSON straight into components. Put it behind your own mock API layer, for example MSW, Next.js API routes, or an in-memory service. That layer should have realistic latency, so your UI works the way it would against a real server. The field names and formats match our real API, so expect to map them into the shapes your UI needs.
- **AI tools** are allowed. If you use them, say where in your README. You must be able to explain every line in the follow-up interview.

What to send us

1. A link to a Git repository (GitHub or GitLab) with a commit history we can follow.
2. A README.md covering:
3. how to run it (ideally with one command),
4. the decisions you made and the trade-offs you accepted,
5. anything you deliberately left out, and what you would do with more time,
6. where you used AI tools, if you did.
7. Tests for the logic you consider most important. We don't expect full coverage.
8. Optional but welcome: a deployed link, a 3–5 minute screen recording walking through it, or both.

How we evaluate

Area	What we look for
UX thinking	Every state makes sense: loading, empty, error, partial, slow and offline. A non-technical customer understands what is happening.
Correctness	The logic is right in the awkward cases in the data, not just the happy path.
Architecture	State lives where it belongs, and a teammate could extend your code without fear.
Accessibility	Keyboard use, screen readers, focus management and contrast.
Performance	It stays smooth with realistic data volume.
Communication	Your README is honest and clear about trade-offs.

In the follow-up interview, you'll walk us through your solution and then extend it live for about 30 minutes.

Domain notes

Providers enter this data by hand. It contains mistakes and inconsistencies. When you find one, decide what the customer should see, handle it, and write the decision down in your README.

Your task

Data file: `data/task3-booking.json`

The story

> "*I live in Kathmandu. I'm booking a check-up at a hospital in Bangkok for when I travel there next month. Which time is the appointment, mine or theirs?*"

Build the date and time picker a customer uses to book an appointment. It includes choosing a specialist and rescheduling an existing booking.

Data

The file contains `providers` (each with its `specialists`), `closures`, `bookedSlots` and an `existingBooking`. Excerpt:

```
// providers[]
{
  "id": 48, "name": "Riverside International Hospital", "country": "TH",
  "timezone": "Asia/Bangkok",
  "daySchedules": {
    "monday":   { "startTime": "08:00", "endTime": "20:00" },
    "friday":   { "startTime": "08:00", "endTime": "17:00" },
    "saturday": { "startTime": "09:00", "endTime": "13:00" }
    // … a day that is missing means the provider is closed that day
  },
  "specialists": [
    { "id": "…", "name": "Dr. A. Wongsakul", "speciality": "Cardiology",
      "consultation": { "startTime": "10:00", "endTime": "16:00", "durationInMinutes": 30,
"bufferInMinutes": 10 } },
    { "id": "…", "name": "Dr. N. Rana", "speciality": "Internal Medicine",
      "consultation": { "startTime": "18:00", "endTime": "21:00", "durationInMinutes": 20,
"bufferInMinutes": 0 } }
  ]
}

// other providers' time zones, exactly as our data stores them
" UTC+05:45"   "Nepal Standard Time"   "(GMT+5:45)"   "Asia/Katmandu "   ""

// closures[]
{ "providerId": 48, "dayOffset": 12, "reason": "Public holiday" }

// bookedSlots[]
{ "providerId": 48, "specialistId": "…", "dayOffset": 0, "startTime": "10:40" }

// existingBooking (for the reschedule flow)
{ "bookingId": "BK-20261008-0042", "providerId": 48, "specialistId": "…",
  "dayOffset": 3, "startTime": "11:20", "numberOfPeople": 1 }
```

Notes on the data:

- Every `startTime` and `endTime` is a wall-clock time in the provider's time zone.
- `dayOffset` counts days from today at the provider: 0 is today there, 1 is tomorrow there, and so on. Turn it into real dates when your mock API starts, so the data never goes stale.
- `timezone` is free text typed in by the provider. Some values are IANA names, some are offsets in various formats, one is a Windows-style name, and one is empty. Handle every format you reasonably can. If a value can't be understood, fall back to the provider's country, and document what you did.
- **The slot rule.** A specialist's slots start at `consultation.startTime` and repeat every `durationInMinutes + bufferInMinutes`. The last slot is one that still ends by `consultation.endTime`. For example, 10:00–16:00 with 30 + 10 gives 10:00, 10:40, 11:20 and so on. A slot is offered only if it lies entirely within the provider's opening hours that day.
- Bookings open for the next 90 days only.

The mock API you build

```
GET  /providers/:id                                          -> provider, including specialists
GET  /providers/:id/slots?date=YYYY-MM-DD&specialistId=… -> slots for that date, with isBooked
POST /bookings { providerId, specialistId, date, startTime, numberOfPeople }
                                                             -> 201 { bookingId } | 409 { message }
PUT  /bookings/:id { date, startTime }                       -> 200 | 409
```

Your mock must be able to return 409 on demand, simulating someone else taking the slot first. It must also support artificial latency and failure (for example a 500 or a network error) on demand, through a small dev panel or query flags, so we can see your slow and error states.

Requirements

1. **Calendar for the next 90 days.** Days in the past, days the provider is closed (weekly or a closure) and days with no free slots are visibly unavailable, and the customer can tell why each one is unavailable.

2. **Two clocks.** Times are shown in the provider's local time, since that is where the customer has to be, with the customer's own local time shown alongside. A customer in Kathmandu booking in Bangkok must never be confused about which time is which. Design how you show this.

3. **Slots.**

4. Group them sensibly, for example morning, afternoon and evening.
5. Booked slots are visible but can't be selected.

6. For today, slots that have already started are unavailable. Measure this by the clock at the provider, not the customer's device.

7. **Specialists.** Choosing a specialist narrows the available slots. When a specialist's consultation hours fall partly or entirely outside the provider's opening hours on a given day, only the overlap is bookable. The customer should understand why some of the specialist's advertised hours aren't offered.

8. **Group bookings.** A customer can book for up to 4 people. Booking for N people needs N consecutive free slots with the same specialist. Only offer start times where that is possible.

9. **Someone else got there first.** If the slot is taken between the customer choosing it and confirming (a 409), keep everything they've entered and explain what happened. Then offer the closest alternatives, without making them start over.

10. **Reschedule.** Starting from `existingBooking`, the customer picks a new slot. Make the change from the old slot to the new one unmistakable before they confirm.

11. **States.**

12. Loading: use skeletons for the calendar and the slot grid, not spinners.
13. A day with no slots, or a specialist with no availability in the 90-day window: explain why and offer a way forward, for example another specialist or the next available date.
14. API failure: allow a retry without losing the customer's selections.

15. Slow network and offline: the customer can tell that something is still loading, or that they are offline, and is never allowed to confirm a booking whose result is unknown.

16. **Accessibility.** The calendar and the slot grid can be navigated fully with the keyboard, with sensible arrow-key behaviour, and screen readers announce them correctly.

17. **Tests.** Cover the time-zone and slot logic with unit tests, including the awkward cases you discover.

Hints on what we will look at closely

- How you handle time zones, including every `timezone` value in the file.
- The edges of the day: late-evening slots, and what "today" means at the provider.
- Your consecutive-slot logic when buffers are involved.

Questions?

If something is genuinely ambiguous, make a reasonable decision, write it down in your README and carry on. That is exactly what we'd want on the job. If you're blocked, email us at **[ADD HIRING EMAIL]** and we'll reply within one working day.

Good luck. We're looking forward to seeing how you think.